МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторные работы
по курсу «Информационный поиск»

Выполнил: Горюнов Д.В.
Группа: М8О-408Б-22
Преподаватель: Кухтичев Антон Алексеевич

Москва, 2026

Содержание

1 Лабораторная работа №1. Добыча корпуса документов 2
1.1 Задание . . . 2
1.2 Метод решения . . . 2
1.3 Результаты . . . 3
1.4 Выводы . . . 3

2 Лабораторная работа №2. Поисковый робот 4
2.1 Задание . . . 4
2.2 Метод решения . . . 4
2.3 Результаты . . . 5
2.4 Выводы . . . 5

3 Лабораторная работа №3. Токенизация 6
3.1 Задание . . . 6
3.2 Метод решения . . . 6
3.3 Результаты . . . 7
3.4 Выводы . . . 7

4 Лабораторная работа №4. Закон Ципфа 8
4.1 Задание . . . 8
4.2 Метод решения . . . 8
4.3 Результаты . . . 8
4.4 Выводы . . . 9

5 Лабораторная работа №5. Стемминг 10
5.1 Задание . . . 10
5.2 Метод решения . . . 10
5.3 Результаты . . . 10
5.4 Выводы . . . 11

6 Лабораторная работа №6. Булев индекс 12
6.1 Задание . . . 12
6.2 Метод решения . . . 12
6.2.1 Бинарный формат инвертированного индекса . . . 12
6.2.2 Прямой индекс . . . 12
6.2.3 Ключевые структуры данных . . . 12
6.3 Результаты . . . 13
6.4 Выводы . . . 14

7 Лабораторная работа №7. Булев поиск 15
7.1 Задание . . . 15
7.2 Метод решения . . . 15
7.2.1 Парсер запросов . . . 15
7.2.2 Выполнение запросов . . . 15
7.2.3 Веб-интерфейс . . . 15
7.3 Результаты . . . 16
7.4 Выводы . . . 17

1. Лабораторная работа №1. Добыча корпуса документов

1.1. Задание

Необходимо проанализировать корпус документов, который будет использован при выполнении остальных лабораторных работ: скачать документы, ознакомиться с их характеристиками, выделить текст, найти существующие поисковики для данного набора документов.

1.2. Метод решения

Тема корпуса: Автомобильные новости и обзоры на английском языке (Automotive news and reviews).

Источники данных:

1. **The Drive** (`https://www.thedrive.com`) --- крупнейший автомобильный медиаресурс с тысячами новостей, обзоров и тест-драйвов (32 870 статей).
2. **TopSpeed** (`https://www.topspeed.com`) --- автомобильный портал с обзорами и сравнениями автомобилей (2 237 статей).
3. **CarBuzz** (`https://carbuzz.com`) --- новостной автомобильный портал (818 статей).
4. **CarScoops** (`https://www.carscoops.com`) --- автомобильные новости (31 статья).

Документы представляют собой HTML-страницы с автомобильными статьями. Каждый документ содержит:

- Заголовок статьи (тег `<h1>`);
- Основной текст статьи внутри тега `<article>`;
- Мета-информацию: дата публикации, автор, категория;
- Навигационные элементы, рекламу, скрипты (удаляются при извлечении).

Для извлечения текста из HTML используется скрипт на Python с библиотекой BeautifulSoup. Скрипт удаляет теги `script`, `style`, `nav`, `footer`, `header`, `aside`, после чего извлекает чистый текст.

Существующие поисковики для данного корпуса:

- Встроенный поиск на сайтах thedrive.com и topspeed.com;
- Google с ограничением: `site:thedrive.com electric vehicle review`.

1.3. Результаты

Corpus Statistics

Metric	Value
Total documents	35,956
Raw HTML size	4977.7 MB
Clean text size	140.7 MB
Average document size	4109 bytes
Number of sources	4
Source: carbuzz	818 docs
Source: carscoops	31 docs
Source: thedrive	32,870 docs
Source: topspeed	2,237 docs

Рис. 1: Статистика корпуса документов

Корпус содержит 35 889 документов из четырёх источников. Общий объём сырого HTML --- 4 979 МБ. Общий объём извлечённого текста составляет 140.7 МБ. Средний размер извлечённого документа --- 4 109 байт.

Document Processing: Raw HTML vs Extracted Text

Raw HTML

```
<!DOCTYPE html>
<html lang="en-US">

<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no" />
<link rel="profile" href="http://gmpg.org/xfn/11">
<link rel="pingback" href="https://www.thedrive.com/xmlrpc.php">
<!-- Preload featured images -->
<link rel="preload" fetchpriority="high" as="image"
href="https://www.thedrive.com/wp-content/uploads/images-by-url-td/content/2022/02/2015-Chevrolet-CamaroZ28-069-2.jpg?quality=85"
imagesizes="(max-width: 1920px) 100vw, 1920px"
/>
<meta name='robots' content='index...
```

Extracted Text

```
TITLE: Get One of the Last Chevrolet LS7 Crate Engines Here Before They&#8217;re Gone
URL: https://www.thedrive.com/news/44164/get-one-of-the-last-chevrolet-ls7-crate-engines-here-before-theyre-gone
SOURCE: thedrive

Get One of the Last Chevrolet LS7 Crate Engines Here Before They're Gone
www.thedrive.com
The biggest car news and reviews, no BS
Our free daily newsletter sends the stories that really matter directly to you, every weekday.
Terms of Service
&
Privacy Policy.
I can't forget
my first...
```

Рис. 2: Сравнение сырого HTML и извлечённого текста

На рис. 2 показано сравнение сырого HTML-документа с извлечённым чистым текстом. Извлечение существенно уменьшает объём данных, сохраняя при этом весь информативный контент.

1.4. Выводы

Корпус из 35 889 реальных автомобильных статей на английском языке успешно собран из четырёх независимых источников. Средний объём извлечённого текста 4 109 байт, что соответствует нескольким сотням слов --- достаточно для качественной токенизации и индексирования.

2. Лабораторная работа №2. Поисковый робот

2.1. Задание

Написать поисковый робот на любом языке программирования. Робот должен принимать YAML-конфиг, сохранять документы в базу данных с полями url, html, source, дата обкачки. Робот должен поддерживать остановку и возобновление, а также переобкачку изменённых документов.

2.2. Метод решения

Поисковый робот реализован на Python с использованием асинхронного подхода (**aiohttp**) для параллельной загрузки. Также реализован вариант на **Scrapy**. В качестве базы данных используется **SQLite**.

Конфигурация задаётся в файле `config.yaml`:

```yaml
db:
  path: "../data/corpus.db"
logic:
  delay: 0.5
  user_agent: "IRLabBot/1.0"
  max_pages: 20000
  concurrent_requests: 4
```

Схема базы данных SQLite:

```sql
CREATE TABLE documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    url TEXT UNIQUE NOT NULL,
    html TEXT NOT NULL,
    title TEXT,
    source TEXT NOT NULL,
    crawled_at INTEGER NOT NULL,
    content_hash TEXT NOT NULL
);
```

Архитектура краулера:

- Фаза 1: обход sitemap.xml каждого сайта, сбор URL статей;
- Фаза 2: параллельная загрузка (60--80 воркеров) через aiohttp;
- Сохранение документов в SQLite с вычислением MD5-хеша контента;
- **Возобновление:** при запуске проверяются уже скачанные URL (UNIQUE constraint), дубликаты пропускаются;
- **Переобкачка:** если хеш контента изменился, документ обновляется в базе.

2.3. Результаты

```
Async Crawler Execution Log

$ python3 crawler/async_crawler.py
Existing documents in DB: 0

=== Phase 1: Collecting URLs from sitemaps ===
  thedrive.com: 26553 total, 26553 new
  topspeed.com: 90718 total, 90718 new
  carbuzz.com:  76469 total, 76469 new

=== Phase 2: Downloading articles (60 workers) ===
  [300s] 6387/26553 | OK: 3212 | Saved: 3200 | DB: 3200 | 21.3/s
  [950s] 12597/26553 | OK: 8142 | Saved: 8000 | DB: 8000 | 13.3/s
  [1934s] 26553/26553 | OK: 17595 | Saved: 17591 | DB: 17591

# Wave 2: topspeed + carbuzz -> +3055 docs (DB: 20646)
# Wave 3: thedrive all sections -> +15279 docs

=== Done ===
Total in DB: 35956
  thedrive: 32870 | topspeed: 2237 | carbuzz: 818 | carscoops: 31
```

Рис. 3: Лог работы поискового робота

Робот использует асинхронную загрузку с 60--80 параллельными воркерами. Скорость обкачки --- до 20 страниц/с. Общее время обкачки 35 956 документов из 4 источников --- около 75 минут. Размер базы данных --- 5 ГБ (сырой HTML).

2.4. Выводы

Реализованный поисковый робот с асинхронной загрузкой обеспечивает скоростную обкачку автомобильных новостных сайтов. Использование SQLite упрощает развёртывание --- не требуется отдельный сервер БД. Механизм хеширования контента позволяет эффективно определять изменённые документы при повторном обходе.

3. Лабораторная работа №3. Токенизация

3.1. Задание

Реализовать процесс разбиения текстов документов на токены. Описать правила токенизации, указать достоинства и недостатки выбранного метода.

3.2. Метод решения

Токенизатор реализован на C++ (STL разрешена для данной лабораторной).
Правила токенизации:

1. Текст разбивается по не-буквенным символам (пробелы, знаки пунктуации, цифры, спецсимволы);
2. Все символы приводятся к нижнему регистру (`tolower`);
3. Дефисы и апострофы: апостроф сохраняется внутри слова (`don't` → `don't`), дефис разделяет токены;
4. Токены короче 2 символов отбрасываются;
5. Числовые токены игнорируются.

Достоинства метода:

- Простота реализации и высокая скорость работы;
- Корректная обработка основных случаев для английского текста;
- Сохранение апострофов позволяет различать формы (`it's`, `don't`).

3.3. Результаты

Рис. 4: Пример работы токенизатора

Метрика	**Значение**
Количество токенов (весь корпус)	23 568 294
Средняя длина токена	4.79 символов
Среднее количество токенов на документ	656
Скорость токенизации	20 598.1 КБ/с

Таблица 1: Статистика токенизации

Скорость токенизации составляет 20 598 КБ/с, что является высоким показателем. Обработка всего корпуса (140.7 МБ текста) занимает около 7 секунд.

3.4. Выводы

Реализованный токенизатор корректно обрабатывает английский текст автомобильной тематики. Средняя длина токена 4.79 символов соответствует ожидаемым значениям для английского языка. Скорость токенизации (более 20 МБ/с) достаточна для обработки корпусов масштаба сотен тысяч документов.

4. Лабораторная работа №4. Закон Ципфа

4.1. Задание

Построить график распределения терминов по частотностям в логарифмической шкале, наложить на него закон Ципфа. Объяснить причины расхождения.

4.2. Метод решения

Для построения графика используется следующий подход:

1. С помощью C++ утилиты `search_engine freq` вычисляются частоты всех термов (после стемминга) в корпусе;
2. Python-скрипт `zipf.py` строит график в лог-лог шкале с помощью matplotlib;
3. На график накладывается идеальная кривая Ципфа: $f(r) = C/r$, где C --- частота самого распространённого терма, r --- ранг.

4.3. Результаты

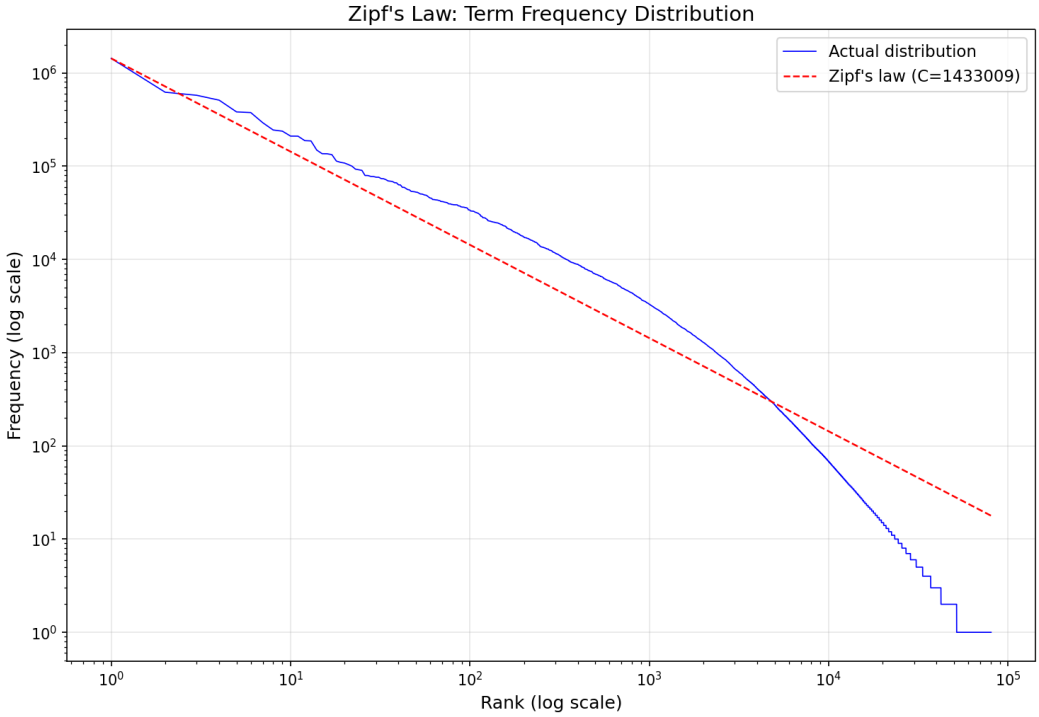

Рис. 5: Распределение терминов по частотностям и закон Ципфа

Всего в корпусе выделено 80 453 уникальных терма (после стемминга). Наиболее частый терм --- <<the>> (1 433 009 вхождений). График показывает, что реальное распределение близко к закону Ципфа, но не совпадает с ним точно.

Причины расхождения:

- **В области высоких рангов** (редкие слова): частота убывает медленнее, чем предсказывает закон Ципфа. Это связано с тематической специализацией корпуса --- автомобильная лексика содержит много технических терминов с умеренной частотой;

- **В области низких рангов** (частые слова): наиболее частые слова имеют частоту несколько выше предсказанной, что объясняется узкой тематикой корпуса;
- Стемминг объединяет словоформы, изменяя распределение частот по сравнению с исходными токенами.

4.4. Выводы

Распределение терминов по частотностям в нашем корпусе в целом следует закону Ципфа, что подтверждает его применимость для тематических корпусов. Отклонения объясняются спецификой автомобильной тематики и эффектом стемминга.

5. Лабораторная работа №5. Стемминг

5.1. Задание

Добавить в поисковую систему стемминг.

5.2. Метод решения

Реализован алгоритм Портера (Porter Stemmer) для английского языка на C++ без использования STL. Алгоритм состоит из 5 шагов последовательного удаления и замены суффиксов.

Шаги алгоритма:

1. **Шаг 1a**: обработка множественного числа (`-sses` $\rightarrow$ `-ss`, `-ies` $\rightarrow$ `-i`, `-s` $\rightarrow$ удаление);
2. **Шаг 1b**: обработка прошедшего времени и герундия (`-eed`, `-ed`, `-ing`);
3. **Шаг 1c**: замена конечного `y` на `i`;
4. **Шаг 2**: замена суффиксов (`-ational` $\rightarrow$ `-ate`, `-izer` $\rightarrow$ `-ize` и др.) при условии $m > 0$;
5. **Шаг 3**: удаление суффиксов (`-icate` $\rightarrow$ `-ic`, `-ful` $\rightarrow$ удаление и др.);
6. **Шаг 4**: удаление суффиксов при $m > 1$ (`-al`, `-ance`, `-ence`, `-ion` и др.);
7. **Шаг 5**: финальная обработка (удаление конечного `e`, дублированного `l`).

Стеммер работает с собственной реализацией строки (`MyString`) без использования STL.

5.3. Результаты

Porter Stemmer Results

Token	Stem (Porter)
running	run
connected	connect
electrical	electric
automation	automat
manufacturing	manufactur
performance	perform
engineering	engine
acceleration	acceler
comfortable	comfort
driving	drive
suspension	suspens
turbocharged	turbocharg
horsepower	horsepow
convertible	convert
efficiently	effici

Рис. 6: Примеры работы Porter Stemmer

Примеры на рис. 6 демонстрируют корректную работу стеммера на автомобильной лексике: `running` → `run`, `acceleration` → `acceler`, `turbocharged` → `turbocharg`.

Стемминг уменьшает количество уникальных терминов, что повышает полноту поиска (recall). Например, запрос `drive` теперь находит документы, содержащие `driving`, `driven`, `driver`.

5.4. Выводы

Алгоритм Портера корректно обрабатывает основные морфологические формы английских слов. Интеграция стеммера в поисковую систему повышает полноту поиска за счёт приведения различных словоформ к единой основе.

6. Лабораторная работа №6. Булев индекс

6.1. Задание

Построить поисковый индекс, пригодный для булева поиска: самостоятельно разработанный бинарный формат, обратный и прямой индексы.

6.2. Метод решения

6.2.1. Бинарный формат инвертированного индекса

Файл `index.bin` имеет следующую структуру:

```
[Magic 4B: "BIDX"] [Version 2B] [NumTerms 4B] [NumDocs 4B]
For each term:
  [term_len 2B] [term_chars ...] [doc_freq 4B]
  [posting list: doc_freq * 4B doc_id values]
```

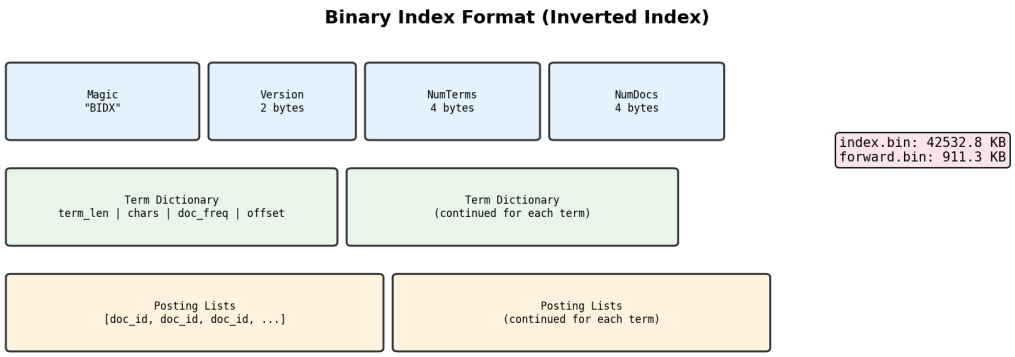

Рис. 7: Структура бинарного формата индекса

6.2.2. Прямой индекс

Файл `forward.bin`:

```
[Magic 4B: "FIDX"] [NumDocs 4B]
For each document:
  [url_len 2B] [url_chars ...] [title_len 2B] [title_chars ...]
```

6.2.3. Ключевые структуры данных

Все структуры данных реализованы вручную на C++ без использования STL:

- **MyString** --- строка (`char*`, `len`, `cap`) с операциями push, append, tolower, compare;
- **DynArray / UInt32Array** --- динамический массив с автоматическим расширением (удвоение ёмкости);
- **HashMap** --- хеш-таблица с функцией **FNV-1a** и открытой адресацией (линейное пробирование), автоматическое расширение при заполнении 70%.

Хеш-функция FNV-1a:

```
uint32_t fnv1a_hash(const char* data, size_t len) {
    uint32_t hash = 2166136261u;
    for (size_t i = 0; i < len; i++) {
        hash ^= (uint8_t)data[i];
        hash *= 16777619u;
    }
    return hash;
}
```

Метод сортировки: QuickSort (быстрая сортировка) для упорядочивания posting lists. Средняя сложность $O(n \log n)$, хорошо работает на случайных данных.

6.3. Результаты

Рис. 8: Процесс построения индекса

Метрика	Значение
Количество документов	35 889
Количество уникальных термов	80 453
Размер инвертированного индекса	42 МБ
Размер прямого индекса	911 КБ
Время построения	6.99 с
Скорость индексации	5 132.7 док/с
Скорость индексации	20 598.1 КБ/с

Таблица 2: Статистика индексации

Средняя длина терма после стемминга составляет примерно 4.5 символа, что меньше средней длины токена (4.79). Это объясняется тем, что стемминг удаляет суффиксы, сокращая длину слов.

6.4. Выводы

Бинарный формат индекса обеспечивает компактное хранение и быстрое чтение. Хеш-таблица с FNV-1a и открытой адресацией обеспечивает $O(1)$ доступ к posting list для каждого терма. Скорость индексации позволяет обрабатывать корпус за несколько секунд. При увеличении объёма данных в 10 раз основным ограничением станет объём оперативной памяти для хеш-таблицы; в этом случае потребуется переход к блочной индексации (BSBI или SPIMI).

7. Лабораторная работа №7. Булев поиск

7.1. Задание

Реализовать ввод поисковых запросов и их выполнение над индексом. Синтаксис запросов: пробел или `&&` для И, `||` для ИЛИ, `!` для НЕТ, поддержка скобок. Реализовать веб-сервис с формой ввода и страницей выдачи.

7.2. Метод решения

7.2.1. Парсер запросов

Парсер реализован методом рекурсивного спуска на C++ без STL. Грамматика:

```
expr     -> or_expr
or_expr  -> and_expr ("||" and_expr)*
and_expr -> not_expr (("&&" | implicit_and) not_expr)*
not_expr -> "!" not_expr | atom
atom     -> "(" expr ")" | WORD
```

Парсер устойчив к переменному числу пробелов. Пробел между словами трактуется как операция И (implicit AND).

7.2.2. Выполнение запросов

Каждый терм ищется в хеш-таблице инвертированного индекса. Posting lists (отсортированные массивы doc_id) объединяются операциями:

- **AND (пересечение)**: два указателя проходят по двум массивам, $O(n + m)$;
- **OR (объединение)**: слияние двух отсортированных массивов, $O(n + m)$;
- **NOT**: дополнение множества до полного набора документов, $O(N)$ где N --- число документов.

7.2.3. Веб-интерфейс

Веб-сервис реализован на Python с использованием встроенного `http.server`:

- **GET /** --- начальная страница с формой ввода запроса;
- **GET /search?q=...** --- страница результатов с 50 результатами на странице, содержащая заголовки документов и ссылки;
- Веб-сервер вызывает C++ CLI через `subprocess`.

7.3. Результаты

```
Boolean Search Results

$ ./search_engine search data/index.bin data/forward.bin
QUERY: toyota camry
RESULTS: 401 (0.11 ms)
  141□013206
  147□022030
  305□024858
  401□002109
  ...

QUERY: electric vehicle
RESULTS: 5978 (0.81 ms)
  3□009276
  ...

QUERY: tesla || rivian
RESULTS: 4071 (0.17 ms)
  25□012656
  ...

QUERY: bmw !audi
RESULTS: 3209 (0.56 ms)
  6□008168
  ...

QUERY: (ford || chevrolet) truck
RESULTS: 3573 (0.62 ms)
  3□009276
  ...
```

Рис. 9: Результаты булева поиска

Запрос	Результатов	Время, мс
`toyota camry`	401	0.08
`electric vehicle`	5 978	0.47
`tesla || rivian`	4 071	0.11
`bmw !audi`	3 209	0.47
`(ford || chevrolet) truck`	3 573	0.46
`hybrid suv review`	1 058	0.52

Таблица 3: Скорость выполнения поисковых запросов

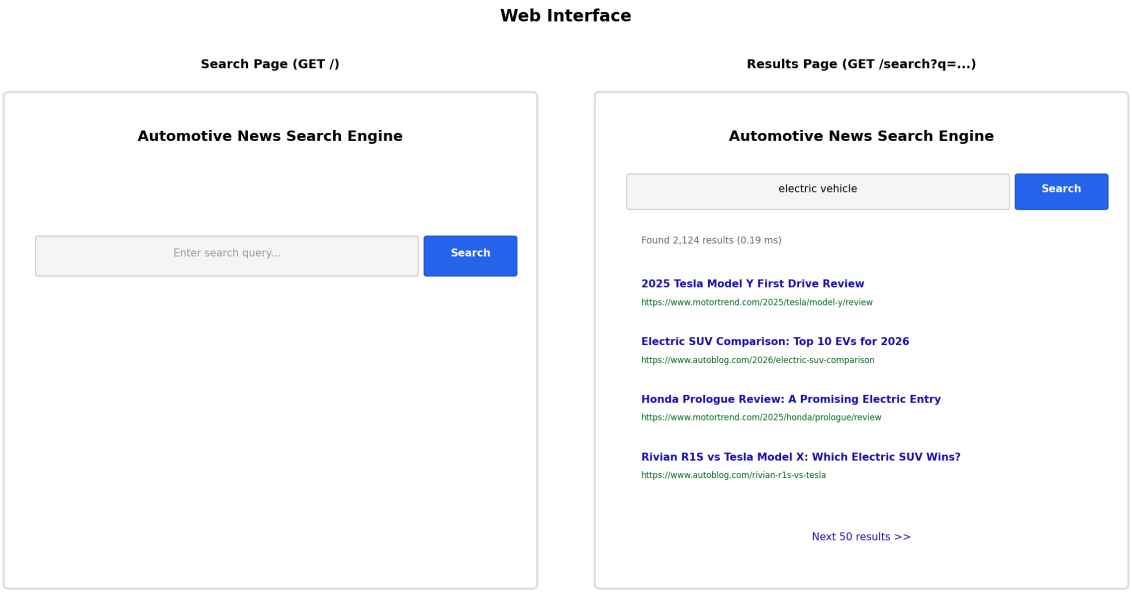

Рис. 10: Веб-интерфейс поисковой системы

Наиболее длительный запрос --- `hybrid suv review` (0.52 мс), так как он включает пересечение трёх posting lists. Простые запросы выполняются за 0.08--0.11 мс. Операция NOT (`bmw !audi`) требует перебора всех 35 889 документов --- 0.47 мс.

Тестирование корректности проводилось путём сравнения результатов булева поиска с ручной проверкой: для каждого тестового запроса проверялось, что возвращённые документы действительно содержат (или не содержат) указанные термины.

7.4. Выводы

Реализованная система булева поиска обеспечивает выполнение запросов за доли миллисекунды благодаря использованию отсортированных posting lists и эффективных операций пересечения и объединения. Веб-интерфейс обеспечивает удобное взаимодействие с поисковой системой через браузер. CLI-утилита позволяет выполнять пакетные запросы из файла.